

Asymptotic Consistent Graph Structure Learning for Multivariate Time-Series Anomaly Detection

Huaxin Pang^{ID}, Shikui Wei^{ID}, Youru Li^{ID}, Ting Liu^{ID}, Huaqi Zhang^{ID},
Ying Qin^{ID}, *Member, IEEE*, and Yao Zhao^{ID}, *Fellow, IEEE*

Abstract—Capturing complex intervariable relationships is crucial for anomaly detection for multivariate time-series (MTS) data. In recent years, graph neural networks (GNNs) have been introduced to explicitly model complex intervariable relationships from global static or local dynamic views, improving the performance of anomaly detection tasks significantly. However, these approaches usually ignore exploring distinct interaction patterns within short context windows or fail to capture unbiased intervariable relationships over longer time windows. To address this limitation, we propose a novel asymptotic consistent graph structure learning (ACGSL) framework for MTS anomaly detection. Specifically, a sequence aggregation module (SeAM) together with a denoising filter is developed to learn the unbiased representation for each temporal variable more effectively. Furthermore, a feature-accumulation graph construct module (FA-GCM) enhanced by asymptotic consistent graph optimization (ACGO) loss is proposed to construct stable interaction graphs over adaptive time windows. We conduct experiments on five benchmarks and achieve remarkable performance enhancement in anomaly detection, even acquiring a maximum gain of 3.64% over the second-best baseline. Furthermore, ACGSL can explicitly give stable intervariable interacted graphs over arbitrary local normal or anomalous states. Extensive experiments and ablation studies demonstrate the effectiveness and robustness of our proposed ACGSL in anomaly detection.

Index Terms—Anomaly detection, deep learning, graph convolution, graph structure learning, multivariate time series (MTS).

I. INTRODUCTION

THERE is now a growing need for security monitoring of many Internet-of-Things (IoT) infrastructures built on cyber-physical systems (CPSs) [1], such as smart factories, cloud servers, and smart transportation [2], due to their high complexity and susceptibility [3]. Accordingly, more and more sensors are being deployed to monitor the behavior of these

IoT systems, and large amounts of time-series data are thus generated [4]. With the advances in big data analysis and deep-learning technologies [5], [6], intelligent models are developed to improve the efficiency of monitoring data processing and provide timely alerts to any potential anomalies to prevent damage from expanding.

The primary challenges in anomaly detection tasks are the availability of labeled data and the diversity of anomalous patterns [7]. As such, unsupervised learning strategies are usually to be considered and used for modeling and training detection frameworks. For univariate time-series data, the existing methods usually rely on the periodicity or trend of time series to detect anomalies, for example, autoregressive moving average (ARMA) [8] and long short-term memory (LSTM) encoder-decoder [9]. However, for these methods, the multivariate time-series (MTS) data consecutively generated by a copious amount of sensors from IoT infrastructures are too complex to model effectively. Deploying multiple monitors of univariate time series in parallel will decrease the speed of the server inference and increase the computing and storing loads. Furthermore, it is noteworthy that time-series data from different sensors are often correlated with each other. Consequently, detecting sensor-specific anomalies of each sensor individually can lead to missed detection or failure to localize the root cause. To tackle this challenge, a series of frameworks adapted to MTS anomaly detection have been proposed. According to the discrepancies of the core backbone, those frameworks can be roughly divided into three categories: recurrent neural network (RNN)-based [10], Transformer-based [7], and graph neural network (GNN)-based methods [11].

Given the impressive performance exhibited in existing GNN-based models, attributed to their proficiency in capturing intervariable relationships, constructing accurate graphs, and effectively propagating contextual influences from neighboring contexts, our framework is established upon GNN-based techniques. For such approaches, constructing graphs in MTS data is important for describing the propagation of relationships between temporal variables. The previous approaches mainly focused on the following aspects, as illustrated in Fig. 1. The first mode [12], [13] calculates the relevant graph via several similarity metric algorithms before training the model. As for the second mode, for example, the graph deviation network approach (GDN) [1] utilizes a graph structure module to optimize the global graph during the training procedure. However, graphs generated by those two modes are framed to

Manuscript received 3 January 2024; revised 29 January 2024; accepted 2 February 2024. Date of publication 26 February 2024; date of current version 4 March 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0112100; and in part by the National Natural Science Foundation of China under Grant U1936212, Grant 62120106009, Grant 52202486, and Grant 62106201. The Associate Editor coordinating the review process was Dr. Yang Song. (Corresponding author: Shikui Wei.)

Huaxin Pang, Shikui Wei, Youru Li, Huaqi Zhang, Ying Qin, and Yao Zhao are with the School of Computer and Information Technology, Beijing Jiaotong University, Beijing 100044, China (e-mail: 20112005@bjtu.edu.cn; shkwei@bjtu.edu.cn; liyouru@bjtu.edu.cn; 20112040@bjtu.edu.cn; yingqin@bjtu.edu.cn; yzhao@bjtu.edu.cn).

Ting Liu is with the School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: liuting@nwpu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIM.2024.3369159>, provided by the authors.

Digital Object Identifier 10.1109/TIM.2024.3369159

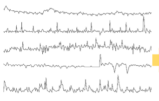
Input Data	Methods	Type (in Test)		Time Span	
		Static	Dynamic	Local	Global
 Multivariate Time Series	Predefined				
	(1) 	✓			✓
	Training				
	(2) 	✓			✓
	(3) 		✓	✓	
	(ours) 	✓			★

Fig. 1. Comparison of different interrelationship graphs constructing methods for MTS data.

be static during the test stage and neglect variable-dependence changes in the short time segment.

In contrast, the deep variational graph convolutional recurrent network (DVGCRN) [14], one of the representative models in the third mode, designs the deep embedding-guided probabilistic generative network to model hierarchical non-deterministic interrelationships based on segmented subsequences. However, learning graphs based on short context windows are biased [15], [16], compared with real interseries dependence. Meanwhile, as MTS has become more data-intensive, arbitrarily enlarging the context window size to learn stable graphs also increases the storage complexity in the inference process. Though DyGraphAD [16] introduces the dynamic time warping distance (DTW) strategy to extend the reception field, which is also insufficient for modeling the long-range time dependence between variables. Thus, how to adaptively model stable and consistent interaction graphs over arbitrary time spans to depict both normal and abnormal patterns remains a serious challenge.

To this end, we propose an asymptotic consistent graph structure learning (ACGSL) framework. It can model the stable interrelationship in MTS data adaptively for detecting anomalies. Specifically, the sequence aggregation module (SeAM) with MTS denoising filters is developed to learn the unbiased temporal embedding for individual variate subsequences after filtering noisy interference effectively. Furthermore, the feature-accumulation graph construct module (FA-GCM) is designed to capture the explicit dependencies in dynamic graphs for each time window. Moreover, to adaptively construct stable interaction graphs over arbitrarily wide time periods, we also propose an asymptotic consistent graph optimization (ACGO) strategy that imposes constraints on the convergence of neighbor graphs to the consistent state. Meanwhile, based on the stable graph, we utilize GNNs to capture the passing interacted information and learn the graph-based representations. Finally, a time-series forecasting decoder makes the one-step-ahead forecast for each variable. Our main contributions are highlighted as follows.

- 1) A novel ACGSL framework for MTS anomaly detection is developed to explicitly model the evolving interrelationship between variables effectively.
- 2) We propose a simple, yet efficient loss, asymptotic consistent loss, to minimize the difference of neighbor

graphs while the values are similar at neighbor time points. Thus, we can model stable interrelationship graphs depicting arbitrary local states.

- 3) Experiments on five benchmarks demonstrate that ACGSL can achieve better anomaly detection performance than known competitive baselines and can model dynamic stable graphs to match the real MTS variation.

II. RELATED WORK

Anomaly detection of MTS has become an active research topic in data science due to its importance in the management and monitoring of intelligent systems [6]. Traditional statistical-based methods [8] have been proven to be effective, but heavily rely on prior knowledge. With the advancement of machine learning, anomaly detection in MTS makes great progress in performance and efficiency [5]. Dong et al. [17] proposed a subsequence time-series clustering-based unsupervised approach, STS-AD, for anomaly detection of the hydraulic system. Omnianomaly [18] utilized a stochastic RNN and a planar normalizing flow to generate reconstruction probabilities. Meanwhile, an adjusted peak over the threshold (POT) [19] was used to realize the automated anomaly threshold selection. CrossFuN [20] designed a time-frequency joint cross-fusion block to capture the relationship between the time and frequency domains and could detect diverse types of anomaly. Recently, Transformers [21], [22] have shown great power in sequential data as the capacity to discover reliable temporal dependencies by the self-attention mechanism. TransAD [7] built the deep transformer network to swiftly perform inference with the knowledge of the broader temporal trends in the data. Anomaly Transformer [23] renovated the self-attention mechanism to the anomaly attention and proposed a mini-max strategy to amplify the normal-abnormal pattern. However, Liang et al. [24] combined generative adversarial networks and autoencoder to build the framework, CCG-EDGAN, achieving anomaly detection and location simultaneously.

As for Graph-based methods [1], [25], their superiority lies mainly in the ability to explicitly model the spatial-temporal dependencies between variates. For instance, MSCRED [15] introduced a multiscale convolutional recurrent encoder and decoder to learn spatial correlations and temporal characteristics in MTS. Grattarola et al. [26] developed a graph autoencoder combining the Riemannian manifolds and adversarial training strategy to detect anomalies. MTAD-GAT [27] employed the graph attention network as a spatial-temporal encoder to learn intervariable and intertemporal dependencies. GDN [1] proposed a graph structure module that learns an underlying stationary graph structure and a graph attention network. Though learning static relational graphs obtains limited gain on anomaly detection, the aforementioned approaches neglect to detect deviation of intervariable associations between normal and anomalous states.

GReLeN [28] was the first to dynamically construct graph structures. It used dynamic graphs to compute the total changes in the in-degree and out-degree values for variable nodes and found the sudden changes promote the robustness for the detection of anomalous events. Dai and Chen [29]

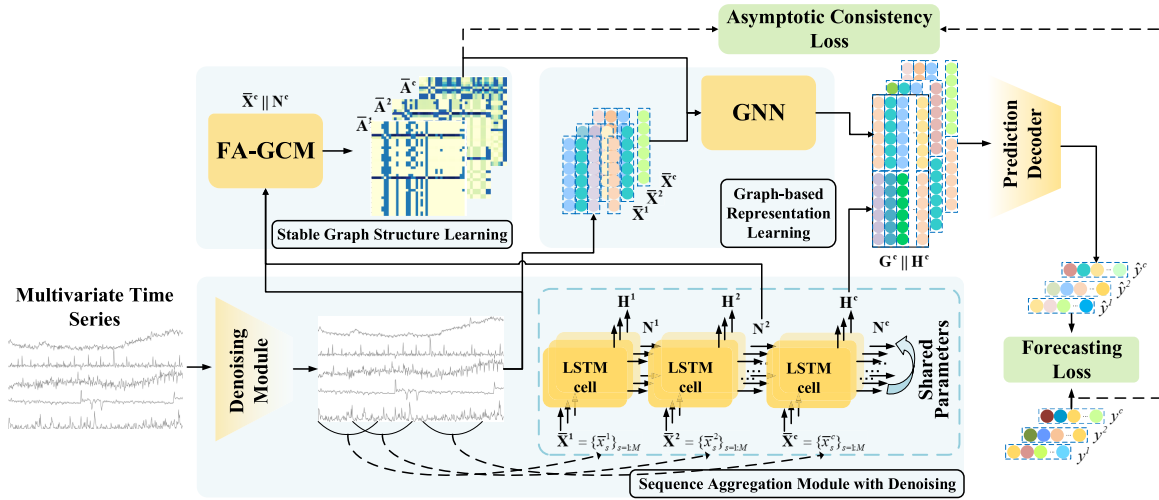


Fig. 2. Overview of our proposed framework. It consists of four components. The *SeAM with denoising* preliminarily filters noisy signals and learns historical features. The *stable graph structure learning* constructs a robust dynamic graph for each sequence. The *graph-based representation learning* uses graph convolution layers to learn the interactive embedding. The *prediction decoder* can forecast the next time-series values.

proposed a graph-augmented normalizing flow model GANF by imposing a Bayesian network among constituent series. Conversely, GIF [30] employs a high-dimensional graph-level embedding method based on the idea of random Fourier features to discover anomalous observations. DVGCRN [14] developed a deep embedding-guided probabilistic generative network to model dynamic interrelationships within MTS to perform the accurate posterior inference. MEGA [31] considered integrating discrete wavelet transform (DWT) into an autoencoder to decompose MTS into multifrequency components. However, existing methods pay less attention to the bias between dynamic graphs of short sequences and local stable graphs reflecting the real dependence of MTS over long-range periods. If feed more long context sequences, time cost will accordingly increase in the inference stage. Consequently, we focus on designing a distinct framework to learn stable and unbiased relational graphs by using short sequences.

III. PRELIMINARIES

A. Model Overview

In this section, we give a brief overview of the proposed framework: At the beginning, raw MTS data are input into the denoising module to eliminate the effect of common noise. Next, the sliced subsequences serve as the input for the SeAM, the graph structure learning module, and the GNN, respectively. Furthermore, the graph-based subsequence embedding and long-time temporal representation are concatenated as the input of the prediction decoder to forecast the latter MTS's values. Moreover, we propose two vital loss functions to jointly optimize the whole model during training. Finally, the hybrid scores are calculated based on the predicted states to detect anomalies at test time. The overall pipeline is shown in Fig. 2.

B. Problem Statement

In this work, we are given an MTS $S \in \mathbb{R}^{M \times T}$, collected over T times ticks with M sensors, as a training set. Note

that M sensors are viewed as M features again. A test set is a time series with M features collected over a different span of time ticks. MTS anomaly detection is defined as a problem that determines whether an observation from a certain task at a certain time is anomalous or not.

X^c is a subsequence extracted from S (S is split into a series of subsequences with stride 1), which represents a collection of time ticks within a sliding context window of length w : $X^c := [S_i, i \in \{c-w+1, c-w+2, c\}]$, where $S_i \in \mathbb{R}^M$ is the MTS at timestamp t . The range of c is $0 \leq c \leq T-1$. Meanwhile, we use x_m^c to present the c th sequence of the sensor m . The work aims at assigning an anomaly score to each time tick in the test set, which is later thresholded to a binary label, with 0 being normal and 1 being abnormal.

IV. METHODOLOGY

A. SeAM With Denoising

MTS datasets collected from real-world scenarios usually unavoidably suffer from intrinsic noise, which detrimentally affects the effective modeling and robust learning of time-series representations. Consequently, most of the existing approaches incorporate noise filtering as a preliminary process, that is, by learning the noise rules in the MTS, then constructing a model to fit the noise values at each moment in time, and finally subtracting the noise from the raw data. This process can be defined as

$$\bar{X} = X - f_D(X; \Theta_D) \quad (1)$$

where $\bar{X}$ denotes the MTS data after filtering the noise and $f_D(X^c; \Theta_D)$ represents the noise-fit function with the parameters Θ_D . Considering that the temporal fluctuations and relational features need to be reserved, we adopt a simple, yet effective MTS denoising module, a linear projection layer with bias, to fit the noise. Specifically, $f_D(X^c; \Theta_D) = \text{Sigmoid}(W_D X^c + b_D)$, $W_D \in \mathbb{R}^{w \times w}$, $b_D \in \mathbb{R}^w$. For each sequence $X^c \in \mathbb{R}^{M \times w}$, the denoised sequence is denoted as $\bar{X}^c \in \mathbb{R}^{M \times w}$. After filtering the noise, we construct a module

aggregating multiple sequence features to learn the long-range temporal embeddings for MTS.

For the anomaly detection task, it is crucial to capture and learn vital information from historical sequences. Inspired by Malhotra et al. [9] and Chen et al. [14], we leverage a stacked LSTM to characterize multilevel and long-range temporal dependencies. Furthermore, an SeAM is designed to integrate relevant historical features for each-variate time series, as follows:

$$H^c, (N^c, O^c) = \text{LSTMs}(\tilde{X}^c, (N^{c-1}, O^{c-1}); \Theta_L) \quad (2)$$

where $N^c \in \mathbb{R}^{M \times d}$ and $N^{c-1} \in \mathbb{R}^{M \times d}$ represent the current c -timepoint and the prior $c-1$ -timepoint output embeddings, respectively. $O^c \in \mathbb{R}^{M \times d}$ and $O^{c-1} \in \mathbb{R}^{M \times d}$ are the hidden stated representations at the corresponding moment. Moreover, $H^c = N^c$, we let H^c serve the predicted decoder, and N^c is employed in the graph-generated process. Θ_L denotes the learnable parameters in the LSTM module. Note that the LSTMs are shared among different time series to reduce learned parameters.

B. Feature-Accumulation Graph Construct Module

A major goal of our framework is to learn the interrelationships between several time series in the form of a stable graph structure. Different from the existing approaches [1], [32], we first design an FA-GCM, which integrates the current denoised sequence features as well as the historical state information to generate a robust dynamic graph. The nodes represent sensors and edges represent dependency relationships between them in the current c th time point. To do this, we define the state matrix $V^c = \tilde{X}^c \| N^c$, where $V^c \in \mathbb{V}^{M \times (w+d)}$ and $\|$ denotes the concatenation operation. According to the state vector $v_i^c \in \mathbb{R}^{w+d}$, the dependent weights can be calculated by the normalized dot product between the embedding vectors of sensors, which can be formulated by

$$e_{ij}^c = \frac{v_i^c v_j^{cT}}{\|v_i^c\| \cdot \|v_j^c\|}, \quad i, j \in \{1, 2, \dots, M\} \quad (3)$$

where e_{ij}^c denotes the correlation value between sensor i and sensor j . All e_{ij}^c form the weighted adjacency matrix A^c . However, with this calculation method, most values of e_{ij}^c are greater than 0, indicating that nearly every sensor has many relations with other sensors. This implies that unrelated sensors might be aggregated with each other. In addition, the dependencies patterns between sensors are not symmetric, that is, e_{ij}^c is not equal to e_{ji}^c . Hence, A^c can be further refined by transforming it into a directed graph, retaining only the key edges

$$\bar{A}_{ij}^c = 1, \quad \{j \in \text{TopK}(e_{ik}^c : k \in \{1, 2, \dots, M\} \setminus \{i\})\} \quad (4)$$

where TopK denotes the indices of top- K values, which is implemented by selecting top k values from the weighted adjacency matrix computed by the normalized dot products. In this manner, we can obtain the refined discretized adjacency matrix $\bar{A}^c$. Note that the value of k is adaptable for each dataset and can be adjusted flexibly by users according to the desired refined level.

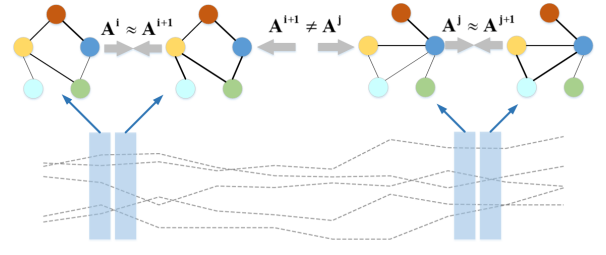


Fig. 3. Schema of the ACGO strategy.

Though we leverage both features $\tilde{X}^c$ of N^c to generate dynamic graphs, these graphs include biased and inadequate relationships caused by the short sequence $\tilde{X}^c$. Besides, existing models cannot address the challenge above. However, we intuitively find that when the moment value of neighbor times is $\|Y^{c+1} - Y^c\|_1 \rightarrow 0$, the relational graphs should be $A^{c+1} \approx A^c$, otherwise $A^{c+1} \neq A^c$. Inspired by the pattern above, we propose a novel and effective strategy, named the ACGO strategy, to constrain neighbor dynamic graphs to keep the same with the guidance of ground truth. We also give a schematic in Fig. 3 to illustrate the core idea of the ACGO. The formula definition is as follows:

$$\min_{\theta} \phi \left(f_{\theta}(X^{c+1}) \cdot f_{\theta}(X^{c+1})^T - f_{\theta}(X^c) \cdot f_{\theta}(X^c)^T \right) \quad \forall \|Y^{c+1} - Y^c\|_1 \leq \sigma \quad (5)$$

where f_{θ} and ϕ denote the feature learning and graph difference calculation, respectively. σ is a tiny constant. We expect that neighbor graphs should be consistent if the discrepancy of labels is less than σ , otherwise, graphs are dynamic and changeable. With ACGO, our method prefers to capture stable dependencies over long-range time series. The implemented details of ACGO are introduced in the model optimization section.

C. Graph-Based Representation Learning

Based on these stable interactions captured by the proposed FA-GCM, sensors can bring to bear on others, which further can change the subsequent behaviors of relevant sensors. To better predict the next-time state of MTS, it is necessary to model the influence from other neighbor sensors for each sensor. Hence, we utilize GNN architecture with parameters Ψ_G to learn the passing influence features and generate the new sequence representation

$$G^c = \text{GNN}(\tilde{X}^c, \bar{A}^c; \Psi_G). \quad (6)$$

To retain the original sequence features $\tilde{X}^c$ at the same time aggregating the influence exerted by other sensors, we employ a simple architecture, graph isomorphism network (GIN) [33]. GIN achieves maximum discriminative power among GNNs and promotes the preserved energy of original nodes by introducing the flexible adjusted variable ϵ . In our work, we employ the single graph convolution layer to integrate the first-order interactions among nodes. Specifically, GIN updates node representations g^c as

$$g_i^c = f_{\Psi}((1 + \epsilon)x_i^c + \text{aggregate}(\{x_j^c, j \in \mathcal{N}(i)\})). \quad (7)$$

Here, multilayer perceptrons (MLPs) are used to model and learn f_ψ , as MLPs can represent the composition of functions. Given the hyperparameter ϵ , we empirically select a fixed scalar, that is, $\epsilon = 0.1$. Furthermore, we utilize the sum mode to aggregate the embeddings of the neighbors $\mathcal{N}(i)$ over node i , according to the researched results [33].

D. Time-Series Forecasting Decoder

Following [7], we build the time-series forecasting decoder to generate the time-series value vector $y^c = s^{c+1} \in \mathbb{R}^M$ at the next time ($c + 1$). Considering that the MTS next-time values are not only relevant to the current features but have a dependency on the historical state, we concatenate the graph-based sequence representation G^c and long-range temporal dependency embedding H^c and then feed this combined representation into the prediction decoder. The forecasting vector $\hat{Y}^c$ is defined as

$$\hat{Y}^c = \sigma(f_2(f_1(G^c \| H^c))). \quad (8)$$

The activated function σ we used is a general ReLU function. Both f_1 and f_2 are two feed-forward networks with the parameter matrices and biases $\{W_1, b_1\}$ and $\{W_2, b_2\}$, respectively. Subsequently, $\hat{Y}^c$ is used to calculate the loss in the training phase and the anomaly score in the test phase.

E. Model Optimization

1) *Asymptotic Consistency Loss*: We used the FA-GCM to build the relation graph for each sequence. The sequences are constantly changing, giving rise to diverse graphs. We propose the ACGO strategy to converge the neighboring graphs to the same and eliminate the disturbance of spurious correlations and noise. To realize the ACGO strategy, we construct the asymptotic consistency loss (ACLoss) function to minimize the variability of the graphs of neighbor sequences by introducing the supervision of ground truth. The ACLoss is defined as follows:

$$\begin{aligned} \mathcal{L}_A^c &= \left[1 - \frac{1}{M} \sum_{i=1}^M (y_i^c - y_i^{c-1}) \right] \cdot \frac{1}{M^2} \sum_{i=1}^M \sum_{j=1}^M (A_{ij}^c - A_{ij}^{c-1}) \\ &= (1 - \|Y^c - Y^{c-1}\|_1) \cdot \|A^c - A^{c-1}\|_1 \end{aligned} \quad (9)$$

where Y^c and A^c present the ground-truth label and weighted adjacency matrix of the sequence X^c , respectively. Similarly, $\{Y^{c-1}, A^{c-1}\}$ are the ones corresponding to the sequence X^{c-1} . Note that we adopt the A^c instead of $\bar{A}^c$ to meet the loss gradient backpropagation. Assuming that $\|A^c - A^{c-1}\|_1$ is a fixed scalar, when $\|Y^c - Y^{c-1}\|_1 \rightarrow 0$, the loss gradually increases, and the model needs to do the large optimization. Otherwise, $\|Y^c - Y^{c-1}\|_1 \rightarrow 1$ means nothing to optimize.

2) *Whole Objective Function*: Since training sets of many datasets only provide the normal data samples, inspired by Tuli et al. [7], our method aims to predict a one-step-ahead forecast based on historical data. Hence, the whole objective function used to optimize our model parameters is defined as

$$\mathcal{L} = \frac{1}{C} \sum_{c=1}^C (\alpha \mathcal{L}_R^c + \beta \mathcal{L}_A^c + \gamma \|A^c\|_2). \quad (10)$$

TABLE I
KEY HYPERPARAMETER SETTINGS IN FIVE DATASETS

Hyper-parameter	SMAP	SWaT	SMD	MSDS	MSL
learning rate	0.001	0.001	0.0001	0.0001	0.008
Epochs	5	6	5	5	5
Top-K	40	30	24	10	5
α	1.0	1.0	0.5	1.0	1.0
β	0.05	0.1	0.01	0.01	0.1
γ	0.001	0.00001	0.001	0.0001	0.001

As the loss $\mathcal{L}_R^c$, we utilize the mean square error (MSE) loss, that is, $\mathcal{L}_R^c = (1/M) \sum_{m=1}^M (\hat{y}_m^c - y_m^c)^2$, to minimize the error between the forecasting value and the ground truth. In addition, $\|A^c\|_2$ is a regularization constraint to graphs via the l_2 norm. α , β , and γ are three hyperparameters to realize the tradeoff between these losses.

V. EXPERIMENTAL RESULTS AND ANALYSES

A. Datasets and Evaluation Metrics

We use five publicly available datasets to evaluate our model performance. The *soil moisture active passive* (SMAP) and *Mars Science Laboratory rover* (MSL) [10] datasets are published by NASA. The *secure water treatment* (SWaT) [34] dataset is released by iTrust Center to support the cyber-attack investigation. The SMD [18] is a *server machine dataset* collected from a large internet company. Moreover, the MSDS [35] dataset is recorded from a complex distributed system for AI operation monitors. The details of their description and preprocessing are found in the Supplementary Material.

To compare the performance between our model and other baselines, we used the evaluation metrics that are used in experiments. We use precision (P), recall (R), $F1$ score ($F1$), and area under the receiver operating characteristic curve (AUC) on the test data and its ground truth to measure the capability of models. Following the prior works [7], [16], [36], the widely used POT [19] method is applied to choose the threshold automatically and dynamically as the one that achieves the best $F1$ over the test set. Besides, we follow the point-adjusting strategy suggested in [18], which labels a whole anomaly segment as 1 as long as one of the points within the segment is detected as an anomaly.

B. Baselines

We compare the performance of our proposed framework with several popular anomaly detection methods. These baseline methods are as follows: LSTM-NDT [10], DAGMM [37], OmniAnomaly [18], MSCRED [15], MAD-GAN [38], USAD [36], MTAD-GAT [27], CAE-M [39], InterFusion [40], GDN [1], TranAD [7], CrossFuN [20], DyGraphAD [16], and Anomaly Transformer [23]. We further give detailed descriptions of datasets and baselines in the Appendix.

C. Experimental Settings

We implement our method in PyTorch version 1.8.1 with CUDA 11.4 and deep graph library (DGL) version 0.9.1.

TABLE II

PERFORMANCE COMPARISON OF ACGSL WITH BASELINES ON FIVE BENCHMARKS. AUC: AREA UNDER THE ROC CURVE, $F1$: $F1$ SCORE (%). THE BEST AND SECOND-BEST SCORES ARE HIGHLIGHTED IN BOLD AND WITH UNDERLINES, RESPECTIVELY

Method	SMAP		SWaT		SMD		MSDS		MSL	
	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1
LSTM-NDT	0.8602	78.79	0.714	61.67	0.9671	90.42	0.8013	88.96	0.9532	77.21
DAGMM	0.9885	88.88	0.8436	81.28	0.9954	94.91	0.9013	88.61	0.9716	84.82
OmniAnomaly	0.9889	87.28	0.8467	81.31	0.9946	94.01	0.8982	88.67	0.9782	87.65
MSCRED	0.9821	86.64	0.8433	80.72	0.9921	84.14	0.8943	88.78	0.9807	93.63
MAD-GAN	0.9891	86.54	0.8463	80.65	0.9933	91.5	0.8054	75.78	0.9862	91.69
USAD	0.989	84.19	0.8460	81.43	0.9933	94.95	0.8979	88.29	0.9795	88.22
MTAD-GAT	0.9844	88.80	0.8464	81.09	0.9921	86.83	0.8982	88.35	0.9899	87.68
CAE-M	0.9901	88.27	0.8464	81.01	0.9783	93.67	0.9013	91.15	0.9903	87.33
InterFusion	-	89.14	-	83.01	-	86.22	-	-	-	86.62
GDN	0.9864	85.18	0.8462	81.01	0.9924	83.42	0.9105	89.00	0.9814	95.91
TranAD	<u>0.9921</u>	89.15	0.8491	81.51	<u>0.9974</u>	96.05	0.9061	<u>89.49</u>	<u>0.9916</u>	94.94
CrossFuN	-	90.48	-	95.34	-	<u>96.86</u>	-	-	-	93.98
DyGraphAD	-	94.94	-	92.31	-	<u>96.55</u>	-	-	-	<u>95.92</u>
Anomaly Transformer	0.9941	<u>96.69</u>	0.9876	94.07	0.9866	92.33	-	-	0.9812	93.59
ACGSL (ours)	0.9847	98.83	0.9883	<u>94.89</u>	0.9990	99.43	<u>0.9063</u>	89.67	0.9944	99.56

TABLE III

PERFORMANCE COMPARISON OF ACGSL WITH BASELINES ON FIVE BENCHMARKS. P: PRECISION (%), R: RECALL (%). THE BEST AND SECOND-BEST SCORES ARE HIGHLIGHTED IN BOLD AND WITH UNDERLINES, RESPECTIVELY

Method	SMAP		SWaT		SMD		MSDS		MSL	
	P	R	P	R	P	R	P	R	P	R
LSTM-NDT	85.23	73.26	77.78	51.09	97.36	84.4	99.89	80.12	62.88	99.91
DAGMM	80.69	98.91	<u>99.33</u>	68.79	91.03	99.14	98.91	80.26	73.63	99.97
OmniAnomaly	81.3	94.19	97.82	69.57	88.81	<u>99.85</u>	99.95	79.64	78.48	99.24
MSCRED	81.75	92.16	99.92	67.7	72.76	99.74	99.94	79.83	89.12	98.62
MAD-GAN	81.57	92.16	95.93	69.57	99.91	84.4	99.82	61.07	85.16	99.3
USAD	74.8	96.27	98.77	68.79	90.6	99.74	99.12	79.59	79.49	99.12
MTAD-GAT	79.91	<u>99.91</u>	97.18	69.57	82.1	92.15	99.19	79.64	79.17	98.24
CAE-M	81.83	85.67	96.97	69.57	90.82	96.71	99.08	84.39	77.51	99.98
InterFusion	89.77	88.52	80.59	85.58	87.02	85.43	-	-	81.28	92.7
GDN	74.8	98.91	96.97	69.57	71.7	99.74	99.89	80.26	93.08	98.92
TranAD	80.43	99.99	97.6	69.97	92.62	99.74	99.95	81.17	90.38	99.99
CrossFuN	89.49	91.50	96.49	94.22	97.57	96.17	-	-	92.48	95.55
DyGraphAD	93.39	96.55	94.29	90.42	95.14	98.19	-	-	<u>97.01</u>	94.86
Anomaly Transformer	94.13	99.4	91.55	96.73	89.4	95.45	-	-	92.09	95.15
ACGSL (ours)	98.76	98.13	90.51	99.72	<u>98.96</u>	99.93	99.98	<u>81.29</u>	99.13	99.99

We train them on a server with Intel¹ Xeon¹ Gold 6248R CPU @ 3.00 GHz and 1 NVIDIA Geforce RTX 3090 graphics card. The models are trained using the AdamW optimizer with different learning rates for each dataset and step scheduler with a step size of 0.5. The window size is set as 10. The hidden dimension in the SeAM and FA-GCM modules is 32. We also set the dropout rate as 0.2 in the prediction layer. Considering the diversities of the number of variates (features) and time-series trends in each dataset, we specifically searched for the optimized combination of key hyperparameters for each dataset to get the best performance. These hyperparameter settings are reported in Table I. For the learning rate and max epochs, we refer to the TranAD and further make slight adjustments according to the number of parameters of our model. The selected process of loss hyperparameter combination $\{\alpha, \beta, \gamma\}$ is shown in the experimental section. For our model ACGSL, we conduct experiments five times with different random seed settings and report the mean score of all metrics. Besides, we adopt the parameter settings that are consistent with those detailed in their respective papers

for the compared methods. Our model code is released at <https://github.com/xinhuaxi/ACGSL-for-MTS>.

D. Performance Comparison

We compare the proposed models with several existing methods, and experimental results (AUC and $F1$ score) are reported in Table II and precision (P) and recall (R) scores are in Table III. We can observe that our model ACGSL obtains significant improvements over all datasets of different forecast horizons. For the MSL dataset, ACGSL has better performances on the $F1$ score than OmniAnomaly, TranAD, and GDN, by 13.59%, 4.87%, and 3.81%. Over the AUC score, our model also achieves competitive performances compared to SOAT baselines and made some solid gains even on top of our higher results, such as the gain of 0.071% than the Anomaly Transformer in SWaT. For the precision and recall metrics, our method still achieves competitive performance. Though most of the baselines realize the high recall, the precisions are low, which means that they can detect all anomalies by predicting more states as anomalies. However, ACGSL realizes the lower false positive ratio (i.e.,

¹Registered trademark.

TABLE IV
ABLATION STUDIES OF THE ACGSL IN TERMS OF AUC AND $F1$ SCORES ON FIVE DATASETS

Method	SMAP		SWaT		SMD		MSDS		MSL	
	AUC	$F1$	AUC	$F1$	AUC	$F1$	AUC	$F1$	AUC	$F1$
w/o Denoise	0.9389	92.15	0.9864	94.04	0.9767	95.20	0.8733	83.88	0.9717	93.98
w/o N^c (in FA-GCM)	0.9498	92.43	0.9806	93.02	0.9850	97.32	0.8820	85.34	0.9922	99.78
w/o $\bar{X}^c$ (in FA-GCM)	0.9578	93.36	0.9863	93.97	0.9701	96.99	0.9023	89.45	0.9684	96.74
w/o H^c (in Decoder)	0.9203	91.41	0.9320	91.13	0.9203	99.64	0.8719	86.51	0.9875	99.99
w/o ACLoss	0.9587	95.71	0.9818	93.56	0.9947	99.25	0.8375	82.13	0.9677	99.10
ACGSL	0.9847	98.83	0.9883	94.89	0.999	99.43	0.9063	89.67	0.9944	99.56

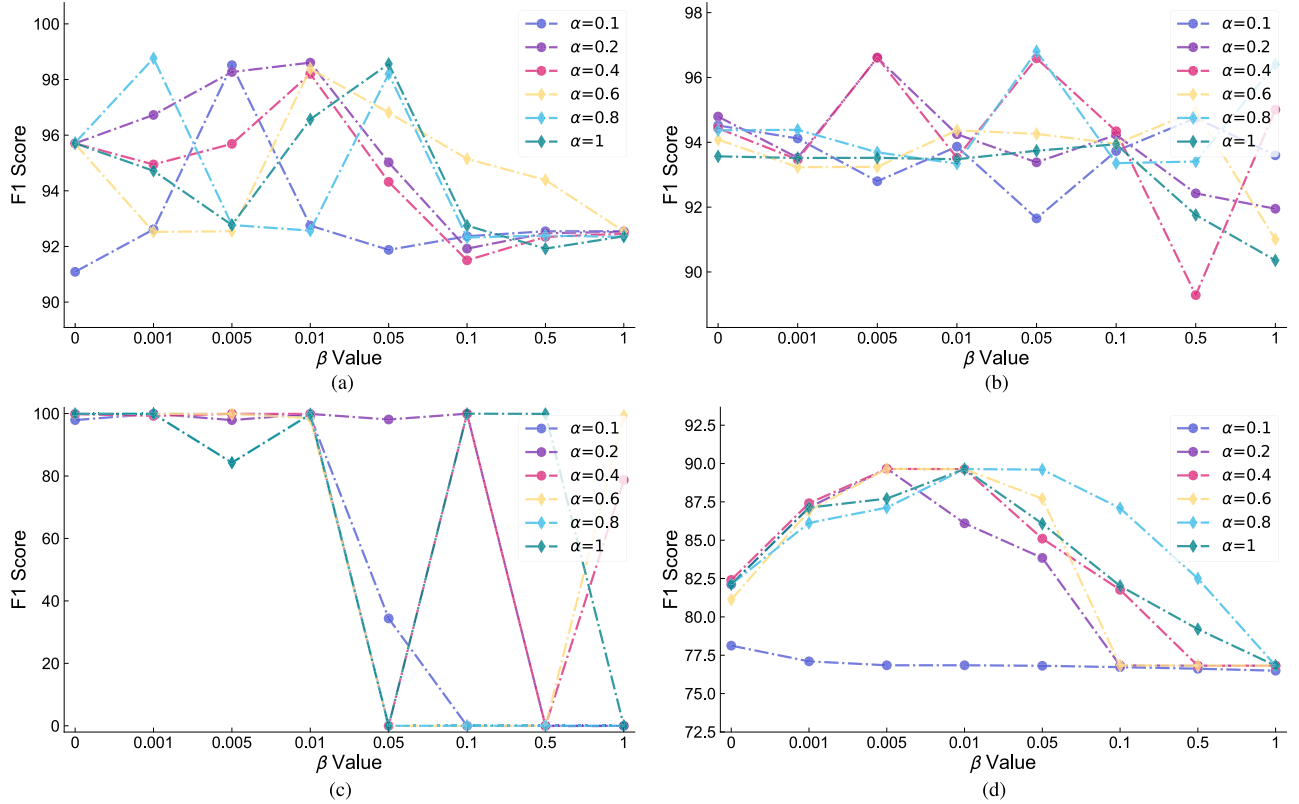


Fig. 4. Empirical analysis for the tradeoff between loss hyperparameters α and β on several datasets. (a) SMAP. (b) SWaT. (c) SMD. (d) MSDS.

1 – precision) while accurately recognizing the presence of multiple types of outliers in the dataset.

Moreover, we summarize some extra views as follows: 1) These methods (LSTM-NDT, MAD-GAN, etc.) obtain poor performances since they adopt the recurrent mechanism to only capture the temporal context dependencies without modeling the relationships among variables. This presents learning the relationships of variables are necessary; 2) Compared to models based on RNN architecture, these approaches (OmniAnomaly, GDN, and MTAD-GAT, etc.) learn static interseries relevances to predict next-step values, neglecting the changes of relationships under the different states. They cannot deal with MTS with changed dependencies well, such as SMAP ($F1 = 88.80\%$, 85.18%) and GDN datasets ($F1 = 86.83\%$, 96.05% for MTAD-GAT and GDN); and 3) DyGraphAD improves the performance by capturing the multiseries relevances in the small-range window. TranAD and Anomaly Transformer use self-attention to capture the dependencies. However, the relationships these models learned

are biased and unstable because of the short sequence input. Our method still outperforms them in terms of AUC and $F1$ as ACGSL uses historical features and ACGSL to generate a dynamic and unbiased graph without the limit of the predefined window.

E. Ablation Studies

To study the necessity of key components of our method, we observe how the model performance degrades by excluding each component: the denoise module, H_h in FA-GCM, X_d in FA-GCM, H_t in the decoder module, and ACLoss, respectively. The intention of removing the denoise module is to figure out whether is necessary to filter extensively existing noise in real-world data. Furthermore, we remove the embedding H_h or X_d to distinguish the dependence of the graph construction process on history features and current features. Note that the effect of the gradual consistency loss is discussed in Section V-F. The results are summarized in Table IV and provide the following findings.

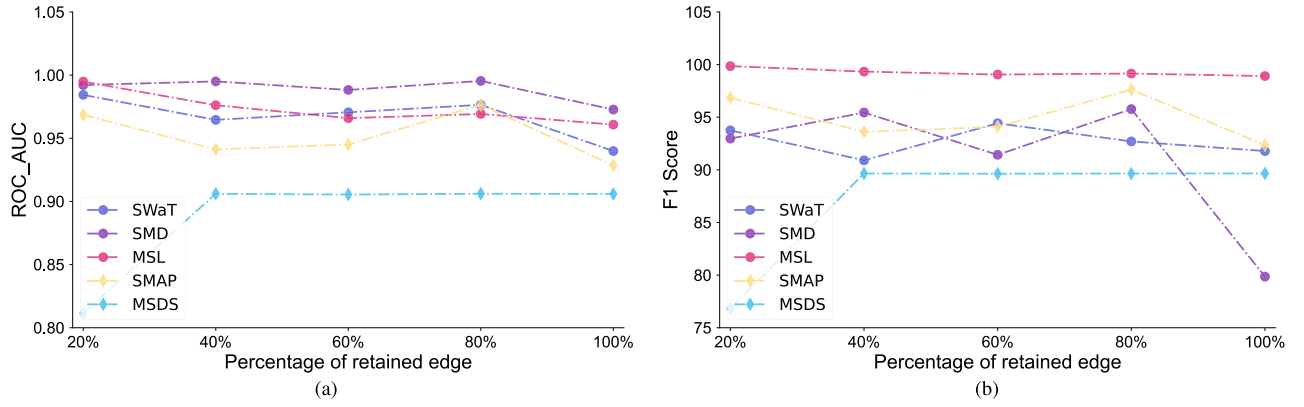


Fig. 5. ROC_AUC score and $F1$ score with percentage of retained edges (top- K) on five datasets. (a) ROC AUC. (b) $F1$ Score.

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT GNN FRAMEWORKS ON FIVE DATASETS

GNN Module	SMAP		SWaT		SMD		MSDS		MSL	
	AUC	F1	AUC	F1	AUC	F1	AUC	F1	AUC	F1
GCN	0.8525	82.70	0.9486	89.88	0.9825	86.32	0.9003	88.95	0.9651	96.65
GAT	0.8396	80.95	0.9847	93.29	0.9038	84.39	0.8086	76.14	0.9673	96.71
GraphSAGE	0.9261	92.12	0.9836	94.37	0.9217	91.21	0.8136	77.10	0.9684	96.74
ChebyNet	0.8932	88.06	0.9810	95.87	0.9972	97.71	0.8086	76.26	0.9684	96.74
GIN (Ours)	0.9847	98.83	0.9882	94.89	0.9990	99.43	0.9063	89.67	0.9944	99.56

- 1) There are significant drops in performance without the denoise module in all datasets. This implies that simply filtering noise is conducive to finding the real state of each sensor and capturing the correct interaction.
- 2) Removing any of H_n and X_d can cause performances degradation. However, the decreased degrees are different in the five datasets. For instance, when we do not employ the H_n in the FA-GCM, the AUC of anomaly detection decreases by 0.0349 over SMAP. On the contrary, there is only a slight change from 0.9944 to 0.9922 in MSL.
- 3) During the decoding process, introducing the embedding H_t containing the historical prior information can help precisely construct the next value, further significantly improving the accuracy of anomaly detection.
- 4) All metric scores realize the significant decrease without using the ACLoss, which presents learning the stable graphs as crucial for the anomaly detection task.

F. Parameter Analyses

1) *Influence of Hyperparameters in the Loss Function:* We conduct comprehensive experiments to find the desired combination between reconstructed loss weight α and gradual consistency loss weight β for optimal convergence with limited epochs. The experimental $F1$ scores are reported in Fig. 4 with the one-to-one correspondence of α and β . When $\beta = 0.0$, the performances have a distinct decrease on multiple datasets, presenting that the proposed ACLoss is essential for anomaly detection. We further summarize all results, finding that the ACGSL reaches the optimal performance as the ratio α/β is within the interval $[10, 100]$. Interestingly, a surprising observation was made on the SMD dataset [see Fig. 4(c)], where the model's performance experienced a

significant decline when the hyperparameter β exceeded 0.05. This unexpected result could potentially be attributed to model overfitting, thereby highlighting the importance of carefully tuning hyperparameters to avoid such issues.

2) *Selection of Top- K :* Using the top- K strategy is conducive to straightforwardly eliminating noise and dispensable interacted influences and capturing the potential cause-effect among sensors. Here, we design experiments to investigate whether different top- K setups affect the model performance. Since the number of sensors in datasets is various, we adopt the percentage of retained edges to unify the process of selecting top- K . The range of percentage is $\{20\%, 40\%, 60\%, 80\%, 100\%\}$. We can observe in Fig. 5 that each dataset corresponds to a different percentage of edges (top- K). For instance, we only reserve the 20% of edges for each sensor (top- $K = 5$) to realize the high AUC score on the MSL dataset. In contrast, the SMAP dataset needs 80% of interactions (top- $K = 40$).

3) *Selection of GNN:* Generally, given the fixed graph and nodes' embeddings, using different GNN modules can construct different nodes' representations. Therefore, to explore which is suitable for anomaly detection, we select several classic GNNs, such as GCN [41], GAT [42], GraphSAGE [43], ChebyNet [44], and GIN [33]. The results of AUC and $F1$ scores are reported in Table V. We can observe that the performances using GIN are superior to other GNN modules on multiple datasets, where the $F1$ score promotes 22.09% than the GAT module on the SMAP dataset. This is because GIN can reserve more original information during the message passing.

G. Case Studies

1) *Learned Graph Visualization:* To investigate whether learned graphs exist the progressive consistency, we conduct

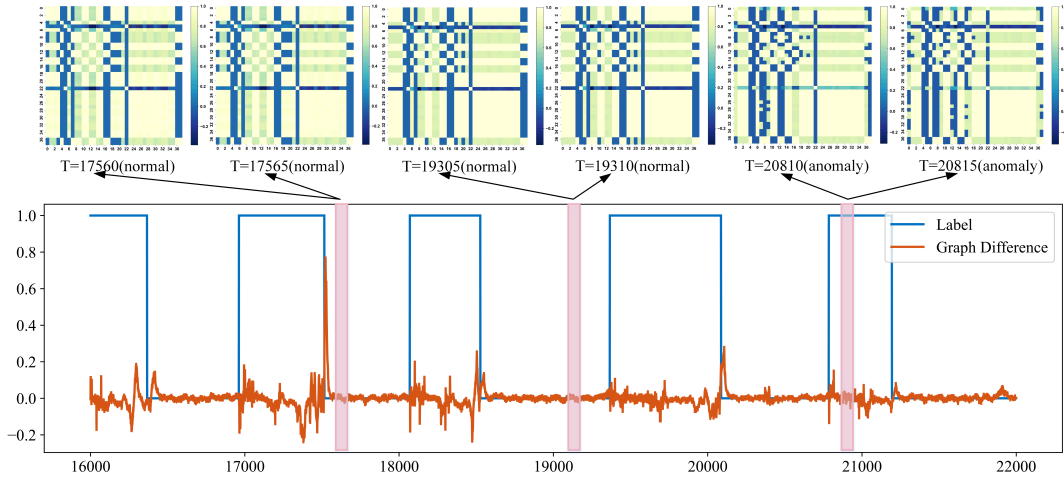


Fig. 6. Empirical analysis for learned graphs on the SMD test set. The top row presents heat maps of learned adjacency matrices at different times. The bottom row shows the relation between anomaly label sequence and graph difference sequence.

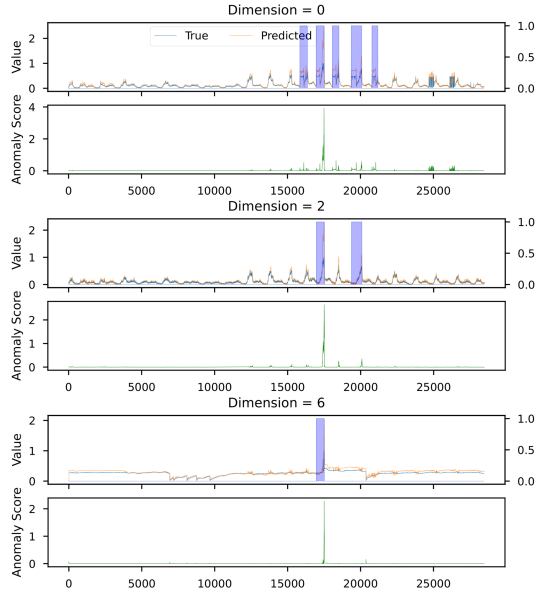


Fig. 7. Predicted and ground-truth labels for the SMD test set, where each dimension denotes the specific sensor. The blue bands present the true anomaly.

a case study by taking the SMD test set as an example. We visualize the interacted graph among sensors at specific timestamps as shown in Fig. 6. Edges in learned graphs provide interpretability by indicating which sensors are related to one another. Besides, the attention weights further indicate the importance of each node's neighbors in modeling the node's behavior. The results explicitly indicate that when the interval time $\Delta_t = 5$, their graphs are similar generally. Especially, these vital edges are still changeless. Moreover, to explore the differences of contiguous timestamps on the whole test set, we calculated the graph difference degree: $\Delta_i = \text{Mean}(G_i - G_{i-1})$. We can find that when the labels are the same, the Δ_i is close to zero. This means that ACGSL learned stable and dependable relations. As labels happen to change, the graph difference degrees are increased, which indicates the graph structure is adjusted. This presents that ACGSL can adaptively capture dynamic interaction among sensors.

2) *Predicted Results Analysis:* We compared the predicted and observed sensor values in Fig. 7 to intuitively evaluate the capability of ACGSL in learning normal patterns of complex MTS. The regions highlighted in blue in Fig. 7 exhibit considerable spikes in anomaly regions. For different sensors, our model can learn the correct change paradigm and precisely construct the values based on the interaction among sensors and historical data. These values of special abnormal locations are also predicted well. Accurate predictions are beneficial to timely finding anomalies and further analyzing the reason behind the phenomena.

VI. CONCLUSION

In this work, an ACGSL framework is proposed for MTS anomaly detection. To model the unbiased and stable intervariable relationship graphs of local normal and anomalous states, we develop two task-oriented modules and one significant loss function. The SeAM with denoising not only filters the noise of the sequence data, but can also learn long-range temporal embedding containing historical semantics. The FA-GCM integrates the context features to generate robust dynamic graphs. More importantly, the asymptotic consistent loss efficiently enforces gradual consistency among adjacent dynamic graphs. It helps ACGSL adaptively model stable and unbiased interaction graphs along any time span and sensitively capture the difference between normal and anomalous signals. Experiments on five datasets have shown the effectiveness of the proposed method in anomaly detection tasks. ACGSL realizes the best performances on several datasets over $F1$ score (98.83% on SMAP, 99.43% on SMD, and 99.56% on MSL) and AUC metrics (0.9883 on SWaT, 0.9990 on SMD, and 0.9944 on MSL). Additionally, unbiased intervariable relationship graphs can help us intuitively understand the changes during abnormal periods. For future work, the more complex denoising architecture, hyperparameter selection, and high-order asymptotics will be further explored.

REFERENCES

- [1] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI Conf.*, 2021, vol. 35, no. 5, pp. 4027–4035.

- [2] D. Lee, Y. Gu, J. Hoang, and M. Marchetti-Bowick, "Joint interaction and trajectory prediction for autonomous driving using graph neural networks," 2019, *arXiv:1912.07882*.
- [3] S. Madakam, R. Ramaswamy, and S. Tripathi, "Internet of Things (IoT): A literature review," *J. Comput. Commun.*, vol. 3, no. 5, p. 164, 2015.
- [4] D. M. Hawkins, *Identification of Outliers*, vol. 11. London, U.K.: Chapman & Hall, 1980.
- [5] S. Thudumu, P. Branch, J. Jin, and J. J. Singh, "A comprehensive survey of anomaly detection techniques for high dimensional big data," *J. Big Data*, vol. 7, no. 1, p. 42, 2020.
- [6] M. Mohammadi, A. Al-Fuqaha, S. Sorour, and M. Guizani, "Deep learning for IoT big data and streaming analytics: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2923–2960, 4th Quart., 2018.
- [7] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, Feb. 2022.
- [8] P. J. Brockwell and R. A. Davis, *Time Series: Theory and Methods*. Springer, 1991.
- [9] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," in *Proc. ICML Anomaly Detection Workshop*, 2016, pp. 1–5.
- [10] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Söderström, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Min.*, 2018, pp. 387–395.
- [11] M. Jin et al., "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection," 2023, *arXiv:2307.03759*.
- [12] Y. Wu, M. Gu, L. Wang, Y. Lin, F. Wang, and H. Yang, "Event2Graph: Event-driven bipartite graph for multivariate time series forecasting and anomaly detection," in *Proc. CIKM Workshop (AMLTs)*, vol. 3375, 2022, pp. 1–8.
- [13] W. Hu, Y. Yang, Z. Cheng, C. Yang, and X. Ren, "Time-series event prediction with evolutionary state graph," in *Proc. WSDM*, Mar. 2021, pp. 580–588.
- [14] W. Chen, L. Tian, B. Chen, L. Dai, Z. Duan, and M. Zhou, "Deep variational graph convolutional recurrent network for multivariate time series anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 3621–3633.
- [15] C. Zhang et al., "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *Proc. AAAI Conf.*, 2019, vol. 33, no. 1, pp. 1409–1416.
- [16] K. Chen, M. Feng, and T. S. Wirjanto, "Multivariate time series anomaly detection via dynamic graph forecasting," 2023, *arXiv:2302.02051*.
- [17] C. Dong, J. Tao, Q. Chao, H. Yu, and C. Liu, "Subsequence time series clustering-based unsupervised approach for anomaly detection of axial piston pumps," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3512212.
- [18] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 2828–2837.
- [19] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
- [20] Y. Bai, J. Wang, X. Zhang, X. Miao, and Y. Lin, "CrossFuN: Multiview joint cross-fusion network for time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 3532109.
- [21] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inform. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [22] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Comput. Surv.*, vol. 55, no. 6, p. 109, 2023.
- [23] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. ICLR*, Apr. 2022, pp. 1–20.
- [24] H. Liang, L. Song, J. Du, X. Li, and L. Guo, "Consistent anomaly detection and localization of multivariate time series via cross-correlation graph-based encoder-decoder GAN," *IEEE Trans. Instrum. Meas.*, vol. 71, 2022, Art. no. 3504210.
- [25] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2021.
- [26] D. Grattarola, D. Zambon, L. Livi, and C. Alippi, "Change detection in graph streams by learning graph embeddings on constant-curvature manifolds," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 6, pp. 1856–1869, Jun. 2020, doi: [10.1109/TNNLS.2019.2927301](https://doi.org/10.1109/TNNLS.2019.2927301).
- [27] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 841–850.
- [28] W. Zhang, C. Zhang, and F. Tsung, "GRELEN: Multivariate time series anomaly detection from the perspective of graph relational learning," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, Vienna, Austria, 2022, pp. 2390–2397.
- [29] E. Dai and J. Chen, "Graph-augmented normalizing flows for anomaly detection of multiple time series," in *Proc. ICLR*, 2022, pp. 1–16.
- [30] D. Zambon, L. Livi, and C. Alippi, "Graph iForest: Isolation of anomalous and outlier graphs," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–8.
- [31] J. Wang, S. Shao, Y. Bai, J. Deng, and Y. Lin, "Multiscale wavelet graph AutoEncoder for multivariate time-series anomaly detection," *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, Art. no. 2502911.
- [32] W. Xiong and X. Sun, "MGADN: A multi-task graph anomaly detection network for multivariate time series," 2022, *arXiv:2211.12141*.
- [33] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *Proc. ICLR*, New Orleans, LA, USA, May 2019, pp. 1–17.
- [34] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ICS security," in *Proc. Int. Workshop Cyber-Phys. Syst. Smart Water Netw. (CySWater)*, Apr. 2016, pp. 31–36.
- [35] S. Nedelkoski, J. Bogatinovski, A. K. Mandapati, S. Becker, J. Cardoso, and O. Kao, "Multi-source distributed system data for AI-powered analytics," in *Proc. 8th IFIP WG 2.14 Eur. Conf. (ESOC)*. Crete, Greece: Springer, 2020, pp. 161–176.
- [36] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Min.*, 2020, pp. 3395–3404.
- [37] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–19.
- [38] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw. (ICANN)*, 2019, pp. 703–716.
- [39] Y. Zhang, Y. Chen, J. Wang, and Z. Pan, "Unsupervised deep anomaly detection for multi-sensor time-series signals," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 2118–2132, Feb. 2023.
- [40] Z. Li et al., "Multivariate time series anomaly detection and interpretation using hierarchical inter-metric and temporal embedding," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 3220–3230.
- [41] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, Toulon, France, Apr. 2017, pp. 1–14.
- [42] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, Vancouver, BC, Canada, May 2018, pp. 1–12.
- [43] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, pp. 1024–1034.
- [44] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, Barcelona, Spain, 2016, pp. 3837–3845.